Master in Data Science – Semantic Data Management 25/26

Graph Representation Learning Laboratory 3 Report

Davide Volpi
davide.volpi@estudiantat.upc.edu

Raffaele D'Agostino
raffaele.d.agostino@estudiantat.upc.edu

Disclaimer: Unintended Graph Symmetry

The PubMed dataset as delivered by PyTorch Geometric stores every citation edge as a bidirectional pair, so that for every triple ($paper_i$, `cites`, $paper_j$) the reverse ($paper_j$, `cites`, $paper_i$) is also present. This was verified empirically via code: all 88,648 triples exhibit a 100% symmetry rate, meaning that each of the 44,324 real citations has a symmeric counterpart. This is semantically incorrect for a citation network, which is by nature an antisymmetric, acyclic relation: a paper can only cite works that preceded it. We decided not to change the provided dataset, and neither to artificially de-symmetrize the citation network since it is impossible to know which one is the true direction of each citation. We instead proceeded with the analyis of the provided network: all analyses in this report are nonetheless carried out on this symmetric graph, and any conclusion drawn from model comparisons are interpreted accordingly.

A.1 Embedding Retrieval via TransE Vector Arithmetic

Methodology: The TransE model treats the knowledge graph like a continuous spatial map. Every paper has a specific coordinate (an embedding), and a citation acts as a fixed movement in a specific direction (the relation vector). Mathematically, this means starting at a citing paper and moving forward along the "cites" vector lands you near the cited paper ($\mathbf{h} + \mathbf{r} \approx \mathbf{t}$). To find a new paper that cites the exact same works as our target paper, we can reverse this movement in three steps:

1. **Find the Center of the Destinations:** Because our target paper cites multiple works, we first locate all those cited papers on the map and find the exact center point between them (the centroid, $\boldsymbol{\mu}_{cited}$). This represents our average destination.
2. **Walk Backwards to the Source:** If a citation moves us forward along the relation vector ($\mathbf{r}_{\texttt{cites}}$), we can find where a citing paper ought to be by standing at that center point and going backward. Subtracting the relation vector gives us a ideal starting coordinate:

$$\mathbf{e}_{ideal} = \boldsymbol{\mu}_{cited} - \mathbf{r}_{\texttt{cites}}$$

3. **Retrieve the Closest Real Paper:** This ideal coordinate is just an empty spot on the map. To finish, we query the coordinates of all actual papers in the database and retrieve the one that sits closest to this ideal spot (measured by Euclidean distance).

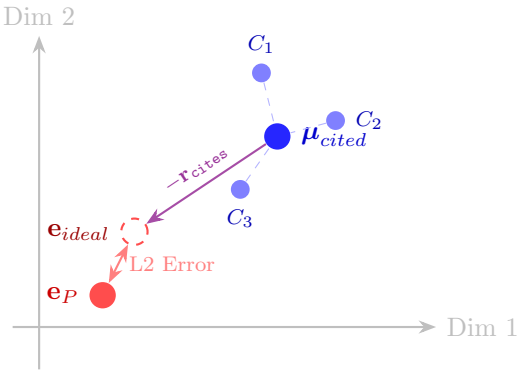

Figure 1: Spatial representation of the retrieval process.

Results: We selected `paper_100` as our target, which cites 2 specific works in the training graph. By finding the center between these 2 works and applying the reverse step, the model calculated the ideal coordinate. Scanning the map for the closest real match, the system retrieved `paper_3332`, with an Euclidean distance of 0.6603 from our ideal spot.

A.2 Structural Limitations of TransE

TransE models citations as a straight movement on a map ($\mathbf{h} + \mathbf{r} \approx \mathbf{t}$). While easy to understand, this rigid geometric rule breaks down when representing the complex reality of citation networks.

1-to-N Problem: Imagine a review paper that cites two completely unrelated works: a math paper and a biology paper.

$$(\texttt{paper\_review}, \texttt{cites}, \texttt{paper\_math}) \quad (\texttt{paper\_review}, \texttt{cites}, \texttt{paper\_biology})$$

TransE is forced to draw the exact same straight-line "cites" arrow from the review paper to both destinations. Geometrically, if you start at the same point and travel in the exact same direction for the exact same distance, you must land in the exact same spot. Therefore, TransE is mathematically forced to conclude that $\mathbf{v}_{\text{math}} \approx \mathbf{v}_{\text{biology}}$, crushing distinct papers into a single point on the map.

Symmetry Problem. In a genuine citation network, symmetry is structurally impossible by definition: a paper can only cite works that preceded it, so the relation `cites` is inherently antisymmetric and TransE would have less difficulty representing it. However, as noted in the Disclaimer, the PubMed dataset as provided by PyTorch Geometric artificially introduces symmetric pairs, so this theoretical non-issue becomes a concrete failure in our setting.

Alternative Solutions: To address these issues, we can use models whose inductive bias is more compatible with a symmetrized citation graph:

- **DistMult:** DistMult uses a symmetric bilinear scoring function, so it is naturally suited to relations for which (h, r, t) and (t, r, h) should receive similar scores. In a graph where every citation edge has been mirrored, this bias becomes advantageous, because the observed *cites* relation behaves as if it were undirected, even though this is not faithful to the original citation semantics.
- **RotatE:** Instead of modeling relations as translations, RotatE represents them as rotations in a complex vector space, and it can capture symmetry, inversion, and composition. This makes

it a viable option in the presence of mutual citations, although in the fully symmetrized case it may be worse than expected, since every symmetric relations correspond to a restricted subset of its rotation space.

A.3 Training KGEs

To establish a baseline, we trained four Knowledge Graph Embedding models on the PubMed dataset: three translational models (TransE, TransH, RotatE) and one semantic model (DistMult). All were trained for 25 epochs using a fixed random seed (2026).

Model	MRR	Hits@1	Hits@3
TransE	0.0586	0.0000	0.0579
TransH	0.0074	0.0013	0.0046
RotatE	**0.6529**	**0.5878**	**0.6866**
DistMult	0.1552	0.0988	0.1707

Table 1: Performance metrics of distinct KGE models on the PubMed dataset.

Results. As shown in Table 1, **RotatE** outperforms the other models by a large margin (MRR = 0.6529). This result, however, must be interpreted with caution, since it is likely influenced by the artificial symmetrization of the citation graph rather than by genuine generalization.

In particular, for many test triples of the form $(paper_i, \texttt{cites}, paper_j)$, the reversed triple $(paper_j, \texttt{cites}, paper_i)$ is already present in the training set. Because RotatE is explicitly able to model symmetric relations, and because perfect symmetry constrains each relation component to phases $\theta_k \in \{0, \pi\}$, the model can exploit this structure very effectively.

To quantify this issue, we measured how many test triples had a symmetric counterpart in the training set, and found that this occurs for 80% of the test set. Therefore, the reported performance is likely inflated by reverse-pair overlap between train and test, which substantially reduces the difficulty of the link prediction task.

For this reason, the apparent strength of the learned RotatE embeddings should not be taken as direct evidence that they provide semantically rich representations of citation structure. Their usefulness as initialization for downstream GNN models will serve as a proxy for generalization, since the high KGE performance may largely reflect adaptation to an artificially symmetric dataset.

TransE collapses to near-zero performance (MRR = 0.0586, Hits@1 = 0.000), confirming the theoretical prediction: symmetric triples force $r_{\texttt{cites}} \approx 0$, destroying the translational signal entirely.

TransH performs even worse than TransE (MRR = 0.0074), because it is designed to project entities onto relation-specific hyperplanes, gaining expressive power proportional to the number and diversity of relations in the graph. With a single relation, however, there is exactly one hyperplane $\mathbf{w}_{\texttt{cites}}$, onto which *all* entities are projected indiscriminately. The multi-relational mechanism that makes TransH useful never activates, leaving it representationally equivalent to TransE but optimisationally more expensive: the unit-norm constraint on $\mathbf{w}_r$ introduces additional instability without providing any structural benefit, compounding the same symmetry collapse.

DistMult achieves a moderate result (MRR = 0.1552). Its scoring function is symmetric by construction, $f(h, r, t) = f(t, r, h)$, so it neither suffers from the directional collapse that destroys TransE and TransH, nor gains from directionality. It nonetheless falls short of RotatE maybe because it cannot model compositional patterns along citation chains, and surely because it has no mechanism to directly exploit the inverse-triple leakage in the test set.

Even though we suspect that the great performance of RotatE are highly inflated by the phenomenon of artificial symmetrization which we discussed eralier (and which can be seen as sort of train-test data leakage), we decided to keep it as the baseline model for the subsequent experiments, since it apparently had the best performances in terms of MRR.

Hyperparameter Grid Search: To explore RotatE performances, we tested 27 different configurations by varying three parameters: Embedding Dimension (32, 128, 256), Negative Samples (1, 5, 15), and Loss Margin (0.5, 1.0, 5.0). Table 2 shows the best and worst combinations.

Rank	Dimension	Neg. Samples	Margin	MRR
Top 3 Configurations				
1	256	15	0.5	**0.7867**
2	128	15	0.5	0.7708
3	256	15	1.0	0.7684
Worst 3 Configurations				
25	32	15	5.0	0.0965
26	32	5	5.0	0.0517
27	32	1	5.0	0.0270

Table 2: Summary of grid search results for the RotatE model, highlighting the extremes of hyperparameter influence.

Analysis of Parameter Effects: The grid search showed how each parameter controls the model:

- **Embedding Dimension:** Dimension 256 dominated, while 32 failed. The PubMed graph is highly complex. A low dimension acts like a bottleneck, simply not giving the model enough geometric space to map out all the distinct nodes accurately.
- **Negative Samples:** Increasing negative samples to 15 improved performance. The model learns by comparing true citations to fake ones. Giving it 15 fake examples per real one forces it to learn much accurate boundaries.
- **Loss Margin:** A small margin (0.5) allowed the model to converge optimally. A massive margin (5.0) caused the model to collapse (MRR 0.0270). A margin of 5.0 is too aggressive: it forces the model to push nodes too far apart that the math breaks down and it stops learning meaningful relationships.

By finding the optimal configuration (Dimension 256, 15 Negative Samples, Margin 0.5), we increased RotatE's performance to an MRR of 0.7867, Hits@1 of 0.7545 and Hits@3 of 0.8099

A.4 Negative Sampling Strategies

Negative sampling generates false triples to teach the model what a non-existent relationship looks like. PyKEEN offers two primary strategies to determine whether to corrupt the head or the tail entity of a valid triple: Uniform and Bernoulli sampling.

Calculated Probabilities. Based on the training split, the two strategies assign the following corruption probabilities:

- **Uniform:** $P(\text{corrupt head}) = 0.5000, \quad P(\text{corrupt tail}) = 0.5000$

- **Bernoulli:** $P(\text{corrupt head}) = 0.4681, \quad P(\text{corrupt tail}) = 0.5319$

Interpretation. Uniform sampling ignores the graph structure entirely, acting as a fixed 50/50 coin flip. Bernoulli sampling instead estimates the structural skew of the relation via two statistics: *tph* (average number of distinct tails per head) and *hpt* (average number of distinct heads per tail), assigning:

$$P(\text{corrupt head}) = \frac{tph}{tph + hpt}.$$

As noted in the Disclaimer, the dataset is perfectly symmetric by construction (100% of triples have their inverse present), which implies $tph = hpt$ on the full graph and would yield exactly $P = 0.5000$ for Bernoulli. The observed slight deviation to 46.8%/53.2% arises because the `tf.split()` call distributes triples randomly across partitions, inevitably separating some symmetric pairs between the training and test sets and introducing a minor structural asymmetry in the training split alone. This deviation is statistical noise, not a property of the citation domain. Consequently, Bernoulli and Uniform sampling are effectively equivalent in this setting: neither provides any meaningful structural advantage, since the graph lacks the severe 1-to-N imbalance that Bernoulli is specifically designed to address. Note that in a real citation network, which is a directed acyclic graph, Bernoulli sampling would result in a much more fair negative sampling method.

A.5 The Isolated Node

Training Dynamics: In KG training, valid citations pull related papers closer together in the vector space. If a paper is completely isolated (zero citations in or out), it never participates in a valid triple. Therefore, it never receives a positive update, so it never gets pulled into any specific neighborhood.

However, the model will randomly select this isolated paper to create fake citations during negative sampling. Every time it is used as a fake example, the loss function explicitly pushes it away from the other papers. Because it experiences zero inward pull and constant outward push, the isolated node spends the entire training process simply getting pushed away from the rest of the graph.

2D PCA Visualization: If we were to compress these vectors and plot them on a 2D map using PCA, the normal papers would form dense, structured clusters based on their topological communities. The isolated paper would appear as an outlier. Because it was constantly pushed away by the active nodes and never anchored to a cluster, it would be relegated to the extreme edges of the coordinate system, completely detached from the main clusters, where the random outward pushes happened to leave it.

B Node Classification with Graph Neural Networks

B.1 Baseline Feature-Based Classification

For this task, we discarded all the edges (`edge_index`) and treated the classification as a standard tabular machine learning problem, relying exclusively on the 500-dimensional TF-IDF word features of each paper.

We utilized the standard, highly unbalanced semi-supervised split for GNNs: 60 nodes for training, 500 for validation, and 1,000 for testing.

Model Selection and Results: We used FLAML to autonomously search across several classifiers, optimizing for the Macro F1-score to account for the extreme sparsity of the training data.

Algorithm & Optimal Hyperparameters	Test Macro F1
Random Forest | `n_estimators`=31, `max_leaves`=8, `crit`=entropy	**0.7224**

Table 3: Baseline classification performance utilizing only node features (ignoring graph structure).

The optimal configuration was a Random Forest achieving a Test Macro F1 of 0.7224, by utilizing 31 distinct decision trees. The selected hyperparameters reflect the constraints of the dataset: the architecture heavily favored highly constrained, shallow trees (`max_leaves` = 8) and restricted each split to evaluate only a small fraction of the available features (roughly 70 out of 500 words).

This configuration is a direct defense against the curse of dimensionality. With only 60 training nodes scattered across a 500-dimensional space, allowing deep, complex trees would immediately lead to catastrophic overfitting. By heavily constraining the leaves and using information gain to split the data, the model effectively acts as a robust keyword detector, successfully isolating the most discriminative TF-IDF terms for each medical class.

B.2 Exploiting the Graph Structure

To overcome the data sparsity, we implemented a GCN to perform semi-supervised learnin, by allowing papers to share their features with their neighbors via message passing.

Architecture Design: We designed an architecture comprising two sequential `GCNConv` message-passing blocks, followed by a standard PyTorch `nn.Linear` classification head.

1. **Layer 1:** Maps the 500-D TF-IDF features to a 64-D hidden space, aggregating information from direct (1-hop) neighbors.
2. **Layer 2:** Outputs a strict 256-D embedding vector, aggregating a broader 2-hop neighborhood context.
3. **Classification Head:** Transforms the 256-D structural embeddings into the final 3 medical categories.

To prevent the model from simply memorizing the 60 training nodes, we applied ReLU activations and Dropout layers ($p = 0.7$) after each GNN block. This forces the network to learn robust, generalizable topological patterns.

The model achieved a Test Accuracy of 0.7790 and a Test Macro F1 of 0.7717.

This is an improvement over the linear classifier, showing that the GNN successfully leveraged the graph's structure. By allowing unlabeled nodes to infer their category based on the known labels and text features of their 2-hop structural neighbors, the GNN effectively outcomes the sparse training bottleneck.

B.3 Tricking the GNN

Because GNNs rely on averaging features with neighbors, they are structurally vulnerable to topological attacks, even if the actual text of the paper remains perfectly intact.

The Strategy: To force the model to misclassify a 'Type 1 Diabetes' paper as 'Type 2' using only 3 fake citations, we can exploit the 2-hop aggregation.

1. **Find the Hubs:** We query the network to find 3 'Type 2' papers that have the absolute highest classification confidence and massive degree centrality.

2. **Inject the Edges:** We add 3 fake citation links connecting our target Type 1 paper directly to these Type 2 hubs.

During the first message-passing layer, our target node pulls in the raw TF-IDF features of the 3 Type 2 hubs. Consequently, during the second layer, those hubs act as a bridge, flooding the target node with the aggregated features of hundreds of other Type 2 papers. This behaviour modify the target node's original Type 1 text features, effectively dragging its final embedding across the decision boundary.

B.4 The more the merrier?

If passing messages is good, is passing more messages better? We tested this by generating GCN architectures with 2, 4, 8, and 16 layers.

Number of GNN Layers	Test Accuracy	Test Macro F1
2 Layers	**0.7630**	**0.7581**
4 Layers	0.7510	0.7457
8 Layers	0.5030	0.4702
16 Layers	0.2760	0.2239

Table 4: Classification performance as a function of Graph Convolutional layers.

The results show a collapse as the network deepens. This is caused by oversmoothing. A 16-layer GNN pulls in data from 16 hops away. Because citation graphs are highly interconnected, a 16-hop radius effectively encompasses almost the entire graph. If every node iteratively averages its features with the entire graph, eventually every single node turns the exact same shade of embeddings. Because the input vectors to the final classification layer become entirely identical, the model completely loses the ability to draw decision boundaries between the classes.

B.5 Operating Without Initial Information

B.5.1 Random Initialization

What happens if we only have the graph edges, but no TF-IDF text features? To test if structural topology alone is enough for a GNN to learn, we replaced the text features with random 256-D noise and retrained the model.

Initialization Strategy	Test Accuracy	Test Macro F1
Standard (TF-IDF Features)	**0.7630**	**0.7581**
Random Initialization (256-D Noise)	0.3830	0.3581

Table 5: Performance degradation when replacing semantic node features with random noise.

The accuracy fell to 38.3%, which is barely better than a random guess (33.3%). GCNs need a meaningful starting signal to propagate. When initialized with noise, the message-passing mechanism simply averages the noise together. With only 60 labeled training nodes, the gradient is far too weak to build meaningful representations out of pure randomness and the structure of the graph.

B.5.2 KGE Initialization

Instead of random noise, we tried initializing the GNN with the pre-trained structural Knowledge Graph Embeddings from RotatE in Task A.3.

Initialization Strategy	Test Accuracy	Test Macro F1
Standard (TF-IDF Features)	**0.7630**	**0.7581**
Random Initialization (256-D Noise)	0.3830	0.3581
KGE Initialization (RotatE 256-D)	0.3800	0.3672

Table 6: Empirical comparison of initialization strategies on predictive performance.

While slightly better than random noise, the KGE initialization failed to recover the performance of true text features. As discussed in the previous sections, we were not expecting an high improvement of performance despite the apparently very good performance of the RotatE model in creating the embeddings. This confirms the hypothesis that the RotatE model was not really learning a good embedding representation of the citations network space, but only memorizing triples from the training set which resulted in good performance when tested on a test set full of their symmetric counterparts.

B.6 Link Prediction with GNNs

Transitioning from predicting node categories to predicting citation links ("Does A cite B?") requires a shift to an Encoder-Decoder architecture.

1. **Pipeline Changes:** Instead of hiding node labels, we hide structural edges (creating train/validation/test edge splits). We must also artificially generate "negative edges" (pairs that do not cite each other) to prevent the model from lazily predicting that everything is connected. Finally, we switch the loss function to a Binary Cross-Entropy.

2. **The Encoder:** Our standard 2-layer GCN is retained, but the final classification head is removed. Its only job is to output the dense d-dimensional embedding for each node.

3. **The Decoder:** We introduce a Dot-Product Predictor. It takes a pair of node embeddings (z_i, z_j) generated by the Encoder, calculates their dot product, and applies a sigmoid function to output a final link probability $P \in [0, 1]$.

B.7 Visualizing Node Embeddings

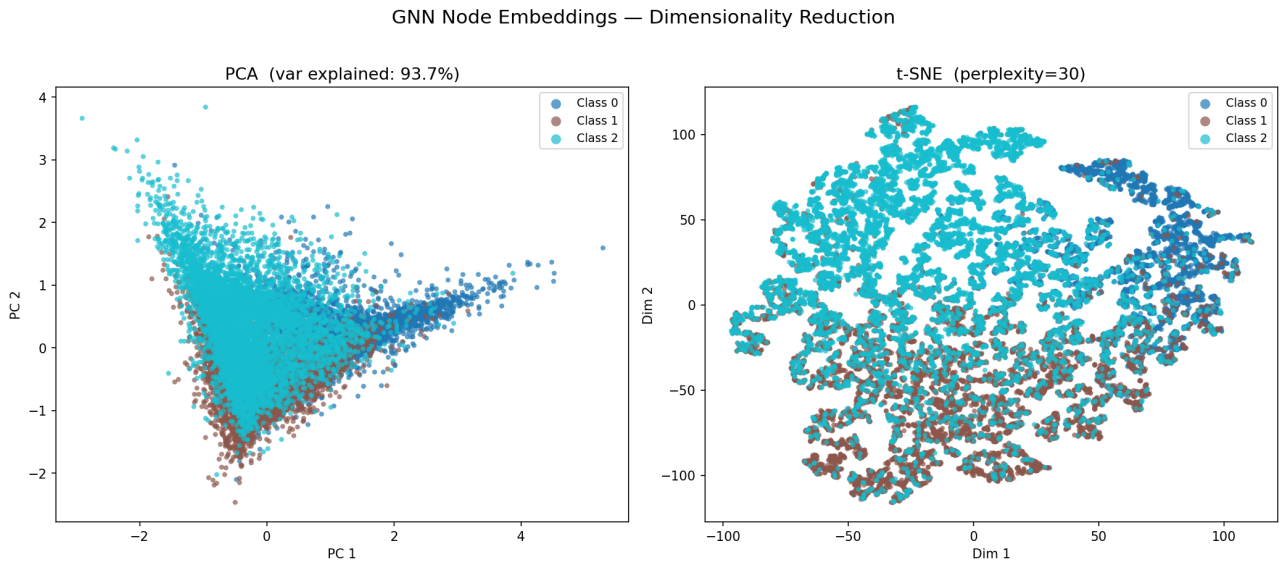

Figure 2: 2D projections of the 256-dimensional GNN node embeddings using PCA (left) and t-SNE (right). Class 0 is Type 1 Diabetes (Blue), Class 1 is Type 2 Diabetes (Brown), and Class 2 is the Other category (Cyan).

To visually verify the network's success, we extracted the 256-D embeddings from the final hidden layer and compressed them into 2D space. **PCA (Linear):** The PCA plot (left) reveals a dense, wedge-shaped distribution. Because PCA is strictly linear, it fails to untangle completely the complex boundaries, resulting in a partial overlap between the three medical classes. Despite it doesn't look as the PCA is able to fully show the separation of classes, it has resulted in a good 93.7% explained variance, meaning that the computed embedding space is separating classes nicely. **t-SNE (Non-Linear):** The t-SNE plot (right), by preserving local neighborhoods using a non-linear decomposition, it reveals nicely separated topological clusters.

B.8 Comparative Analysis of Embedding Spaces

The embeddings produced by KGEs and by GNNs are not the same, even when they refer to the same PubMed papers, because they are learned with different objectives, parameterizations, and inductive biases.

KGE embeddings are optimized to model relational structure and link plausibility in the graph, whereas GNN embeddings are obtained through neighborhood aggregation and are shaped by prediction task.

Therefore, it would not be mathematically sound to create a new embedding space by randomly choosing, for each paper, either its KGE embedding or its GNN embedding. Since the two latent spaces are generally not aligned, such a random mixture would produce representations with no consistent interpretation.

Rank	GNN Neighbor (ID)	Cosine Sim.	KGE Neighbor (ID)	Cosine Sim.
1	10723	0.9999	6069	0.2334
2	8456	0.9999	15325	0.2309
3	18298	0.9999	12341	0.2292
...	...	...	...	...

Table 7: Top nearest neighbors for Target Paper 100. The intersection between the sets is exactly 0.

This is proven empirically in Table 7. The top 10 nearest neighbors for Paper 100 in the GNN space share exactly zero overlap with its neighbors in the KGE space. Moreover:

- **GNN Similarity ($\sim$ 0.9999):** The GNN compresses intra-class variance. Its neighbors are simply other papers with similar text that were dumped into the exact same categorical bucket.
- **KGE Similarity (0.2334):** RotatE spaces nodes based on structural network roles, regardless of text content. Its neighbors are structurally equivalent nodes (e.g., two review papers with similar citation reach), even if they belong to entirely different medical categories.

B.9 Generating Embeddings for New Papers

If a brand-new paper is published today, how do our two models handle it?

GNN (Fast and Generalizable): Graph Neural Networks don't memorize specific nodes; they learn a generalizable "recipe" (the weights) for mixing features with topology. To handle the new paper, we simply append its TF-IDF features to our matrix, add its new edges, and execute a single forward pass. The network then outputs a perfect embedding. It is computationally fast and ideal for real-time recommendation.

KGE (Slow and Rigid): KGE act as a giant lookup table, memorizing a specific, discrete vector for every exact node seen during training. Because KGEs are completely blind to text features, the new paper's TF-IDF vector is useless. We are forced to assign the new paper a random vector, freeze the rest of the graph, and iteratively run gradient descent optimization loops just to slowly drag that new vector into alignment with its neighbors. It is computationally expensive and highly rigid.